

Question Answering

Le Thanh Huong

School of Information and Communication Technology

How can you solve the following problem?

1. Build a QA system based on a set of FAQs/user manual
2. Build a chatbot for a computer store
3. Build a QA system with the knowledge base from Wikipedia
4. Build a QA system for advising education enrollment
5. Build a QA system for study counseling

Question Answering

- **Target:** Build a system that can answer human questions automatically in natural language



- **Sources:**
 - Text passages, web documents, knowledge bases, databases, available Q&A sets
- **Question types:** return a value or not, open/closed domain, simple/complex questions, etc.
- **Answer types:** a few words, a paragraph, a list, yes/no, etc.

Sample TREC questions

- **Who** is the **author** of the book “The Iron Lady: A Biography of Margaret Thatcher”?
- **What** was the **monetary value** of the **Nobel Peace Prize** in **1989**?
- **What** does the **Peugeot** company **manufacture**?
- **How much** did **Mercury** **spend on advertising** in **1993**?
- **Why** did **David Koresh** ask the **FBI** for a **word processor**?

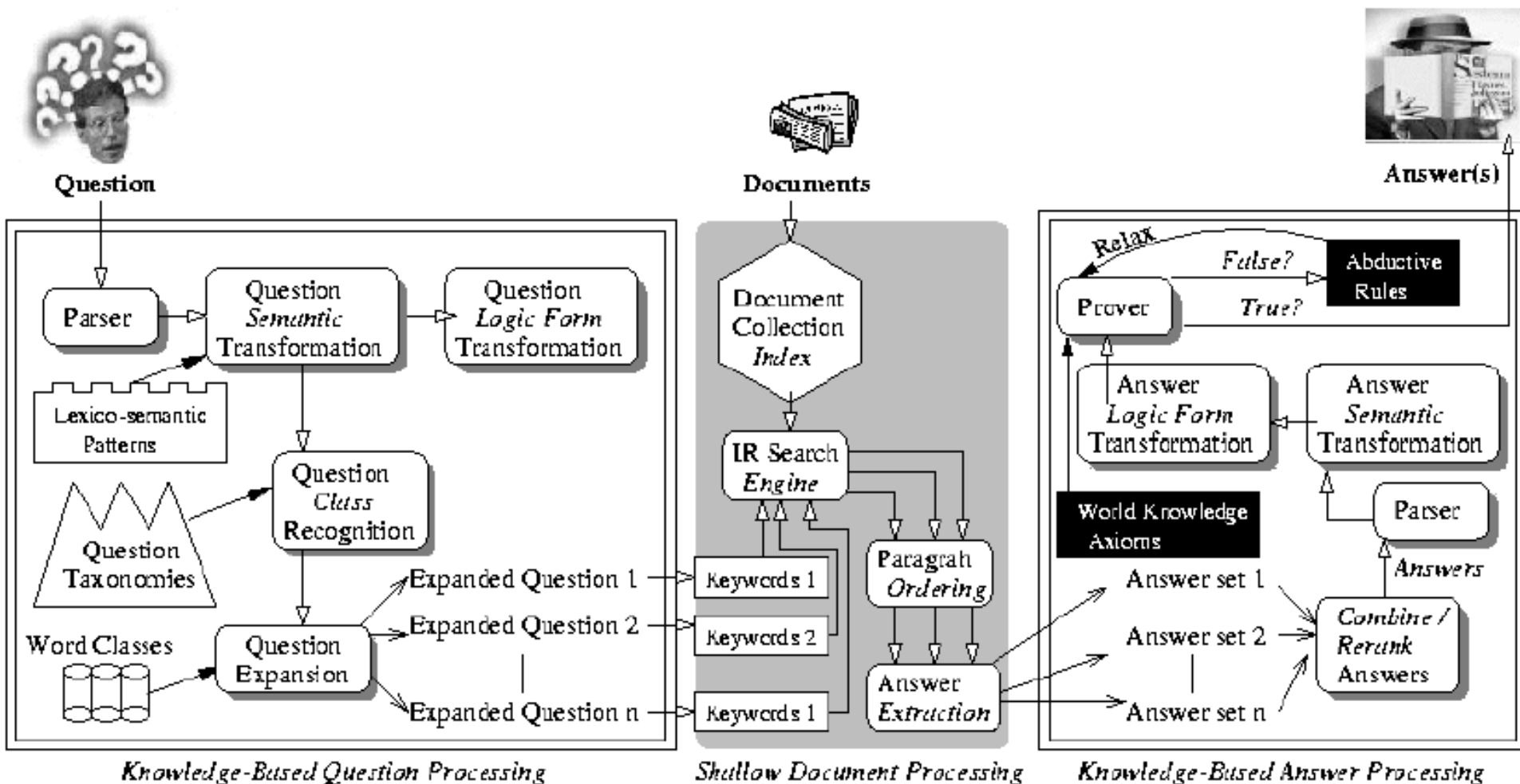
➤ Main tasks:

- Intent detection/classification
- Named entity recognition/Slot filling
- Information Retrieval
- Answer Generation

Other sample questions

- Give me **information** about some **Dell laptops** with prices in the range from **18 million** to **22 million VND**
- Can you **recommend** me some **tourist attraction** in **Hanoi**?
- Give me some **information** about **houses for rent** near **HUST** with the price under **5 million VND**
- **Which city** has the **largest population** in **Vietnam**?

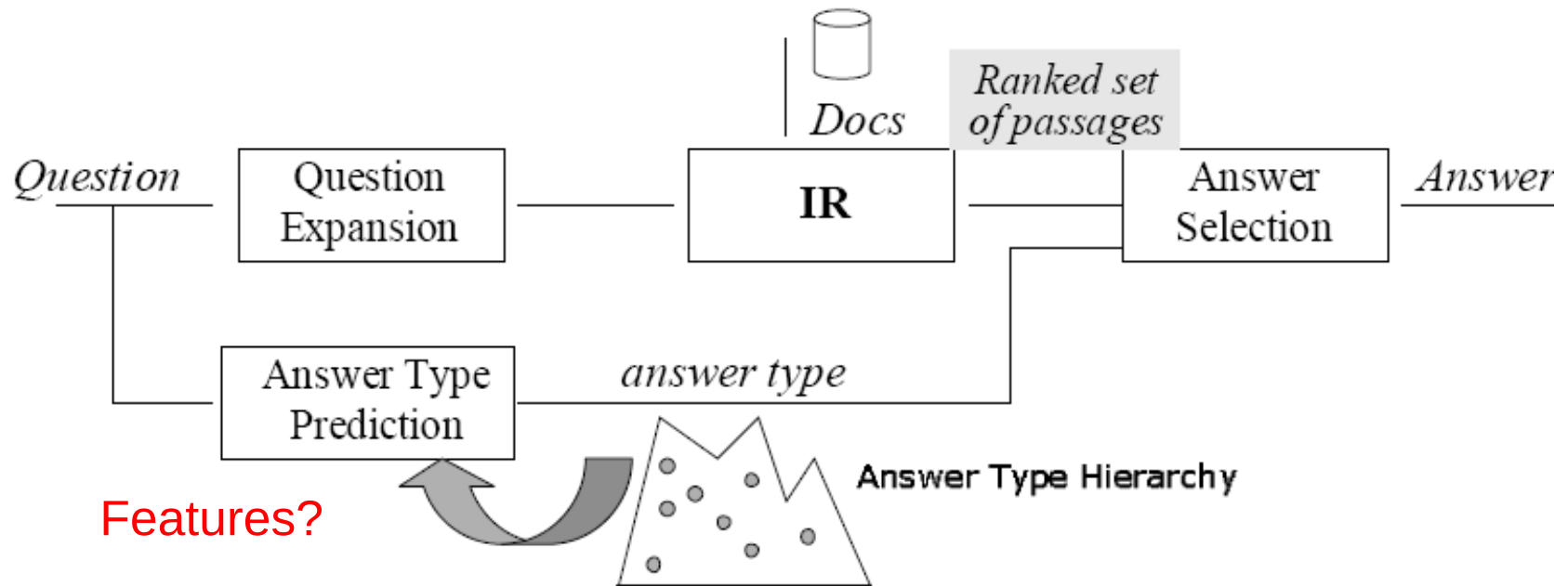
Full NLP QA



Moldovan, Dan I. et al. "LCC Tools for Question Answering." *Text Retrieval Conference* (2002).

Answer types in QA systems

- Answer type:
 - Labels questions with answer type based on a taxonomy
 - Classifies questions (eg., by using a maximum entropy model)



M. Pasca and S. Harabagiu, "High performance question/answering," in *Proc. 24th Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2001

Answer Types

- “Who” questions can have organizations as answers
 - Who sells the most hybrid cars?
- “Which” questions can have people as answers
 - Which president went to war with Mexico?

Keyword Selection Algorithm

Select all...

- Non-stopwords in quotations
- NNP words in recognized named entities
- Complex nominals with their adjectival modifiers
- Other complex nominals
- Nouns with adjectival modifiers
- Other nouns
- Verbs
- The answer type word

Machine reading comprehension (MRC)

Stanford Question Answering Dataset (SQuAD)

- Questions are crowd-sourced.
- 100k annotated (passage, question, answer) triples
- Passages are selected from English Wikipedia, usually 100~150 words.
- Each answer is a short segment of text in the passage.
- SQuAD was the most popular reading comprehension dataset; the SOTA exceeds the estimated human performance.

(Rajpurkar et al., 2016): SQuAD: 100,000+ Questions for Machine Comprehension

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called “showers”.

What causes precipitation to fall?

gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?

graupel

Where do water droplets collide with ice crystals to form precipitation?

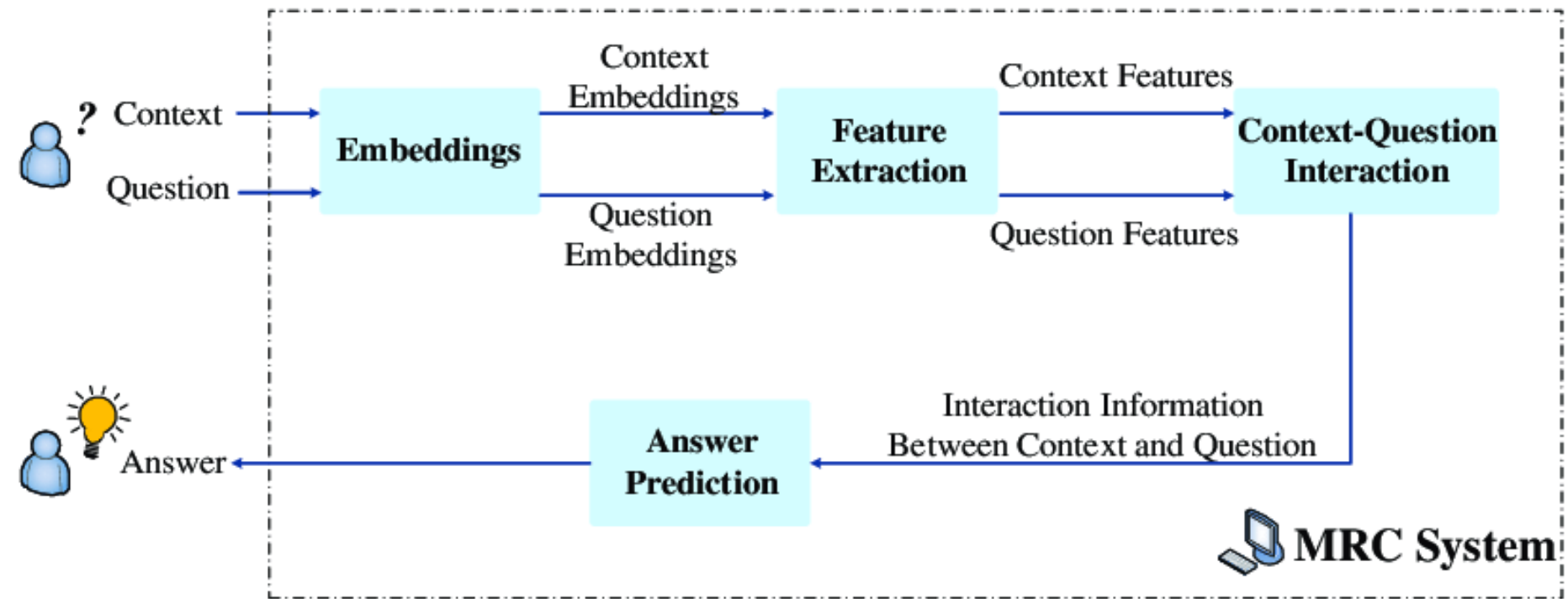
within a cloud

Stanford question answering dataset (SQuAD)

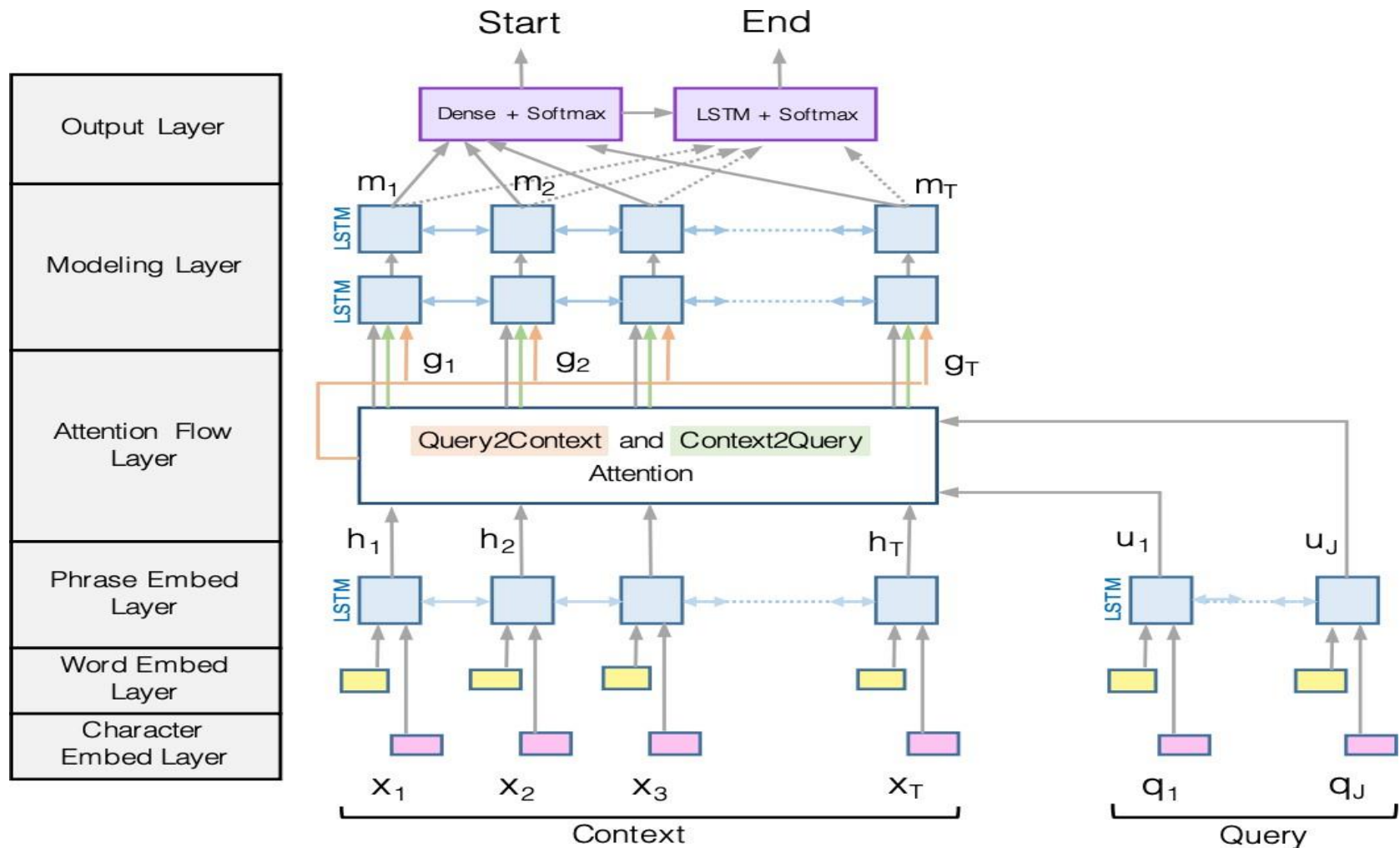
Evaluation:

- Exact match (0 or 1) and F1 (partial credit).
- For development and testing sets, 3 gold answers are collected, because there could be multiple acceptable answers.
- We compare the predicted answer to each gold answer (a, an, the, punctuations are removed) and take max scores. Finally, we take the average of all the examples for both exact match and F1.
- Estimated human performance: EM = 82.3, F1 = 91.2
 - Q: What did Tesla do in December 1878?
 - A: {left Graz, left Graz, left Graz and severed all relations with his family}
 - Prediction: {left Graz and served}
 - Exact match: $\max\{0, 0, 0\} = 0$
 - F1: $\max\{0.667, 0.667, 0.461\} = 0.67$

Neural models for reading comprehension

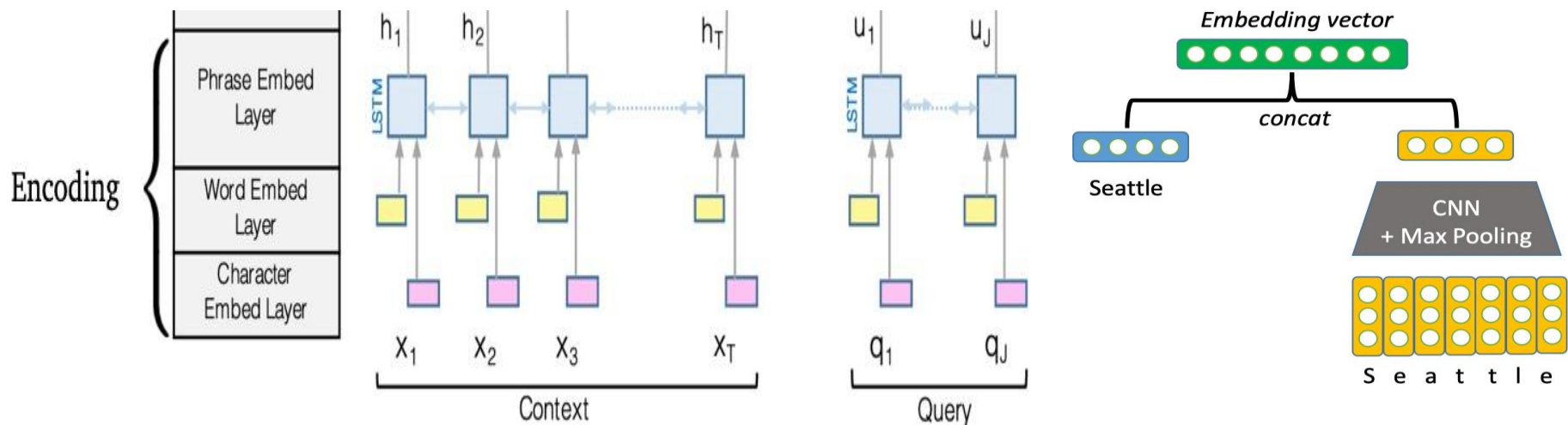


BiDAF: the Bidirectional Attention Flow model



(Seo et al., 2017): Bidirectional Attention Flow for Machine Comprehension

BiDAF: Encoding



- Use a concatenation of word embedding (GloVe) and character embedding (CNNs over character embeddings) for each word in context and query

$$e(c_i) = f([\text{GloVe}(c_i); \text{charEmb}(c_i)]) \quad e(q_i) = f([\text{GloVe}(q_i); \text{charEmb}(q_i)])$$

- Then, use two bidirectional LSTMs separately to produce contextual embeddings for both context and query

$$\vec{c}_i = \text{LSTM}(\vec{c}_{i-1}, e(c_i)) \in \mathbb{R}^H$$

$$\overleftarrow{c}_i = \text{LSTM}(\overleftarrow{c}_{i+1}, e(c_i)) \in \mathbb{R}^H$$

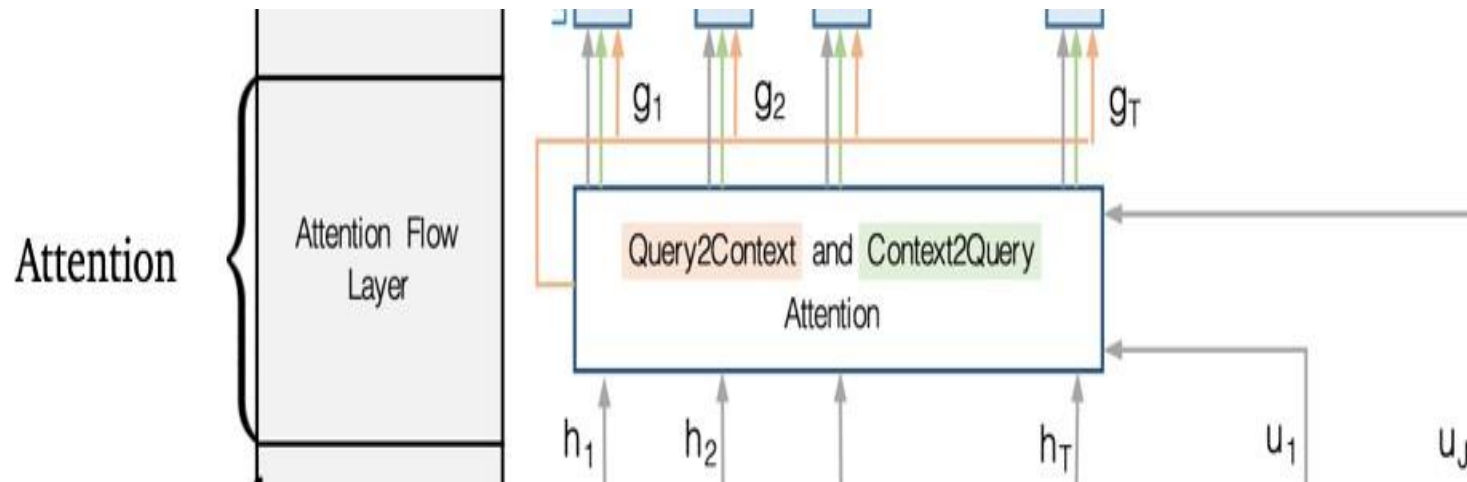
$$\mathbf{c}_i = [\vec{c}_i; \overleftarrow{c}_i] \in \mathbb{R}^{2H}$$

$$\vec{q}_i = \text{LSTM}(\vec{q}_{i-1}, e(q_i)) \in \mathbb{R}^H$$

$$\overleftarrow{q}_i = \text{LSTM}(\overleftarrow{q}_{i+1}, e(q_i)) \in \mathbb{R}^H$$

$$\mathbf{q}_i = [\vec{q}_i; \overleftarrow{q}_i] \in \mathbb{R}^{2H}$$

BiDAF: Attention



- Context-to-query attention: For each context word, choose the most relevant words from the query words

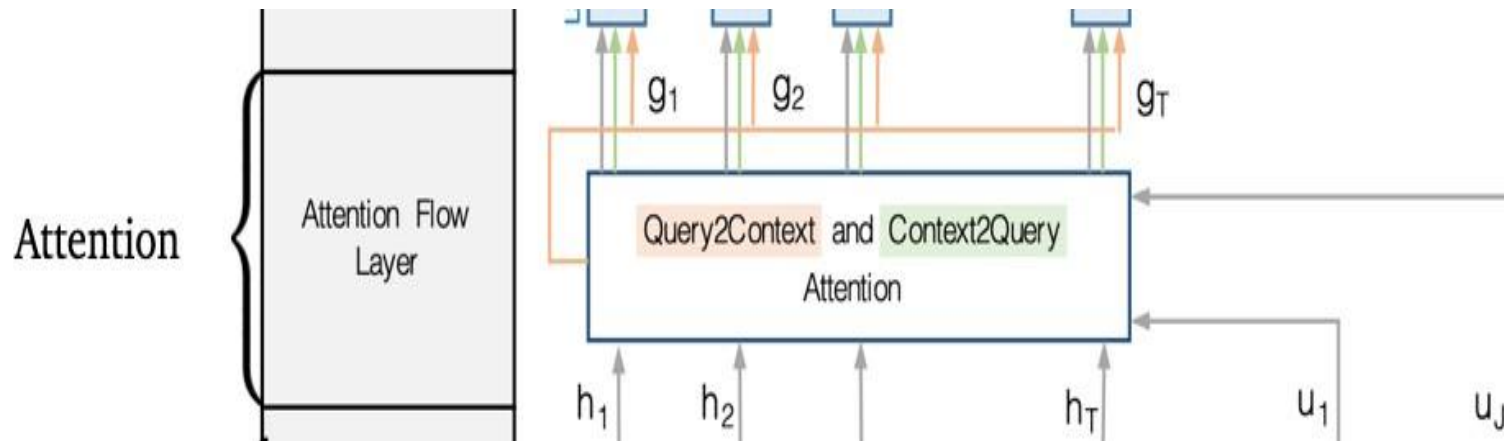
Q: Who leads the United States?

C: Barak Obama is the president of the USA.



(Slides adapted from Minjoon Seo)

BiDAF: Attention



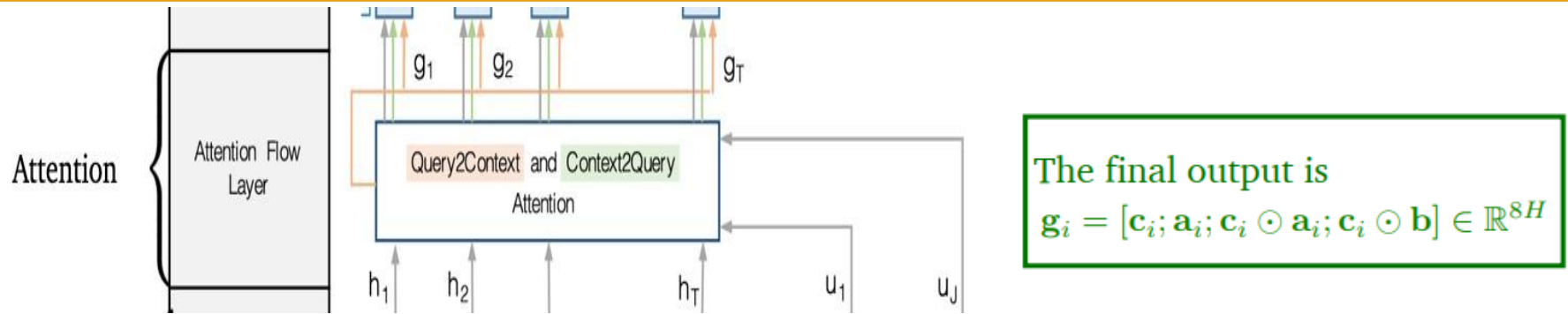
- Query-to-context attention: choose the context words that are most relevant to one of query words

While Seattle's weather is very nice in summer, its weather is very rainy in winter, making it one of the most gloomy cities in the U.S. LA is ...

Q: Which city is gloomy in winter?

(Slides adapted from Minjoon Seo)

BiDAF: Attention



- First, compute a similarity score for every pair of (c_i, q_j) :

$$S_{i,j} = \mathbf{w}_{\text{sim}}^T [c_i; q_j; c_i \odot q_j] \in \mathbb{R} \quad \mathbf{w}_{\text{sim}} \in \mathbb{R}^{6H}$$

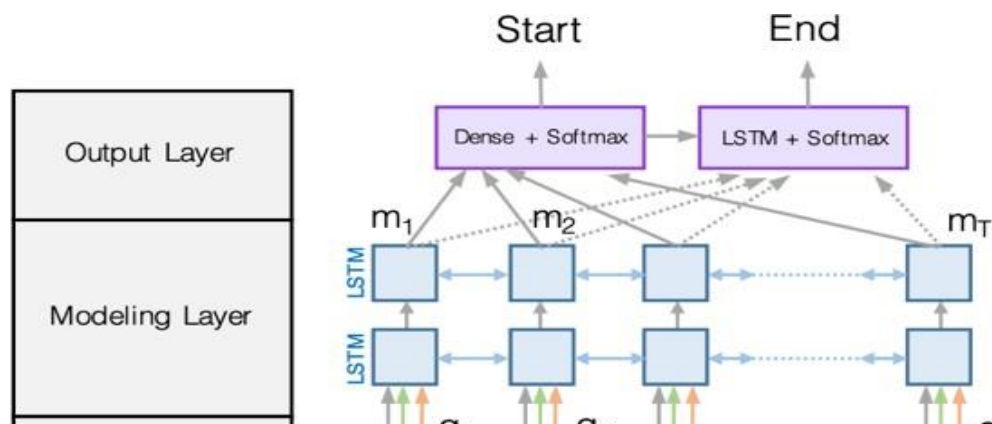
- Context-to-query attention (which question words are more relevant to c_i)

$$\alpha_{i,j} = \text{softmax}_j(S_{i,j}) \in \mathbb{R} \quad \mathbf{a}_i = \sum_{j=1}^M \alpha_{i,j} \mathbf{q}_j \in \mathbb{R}^{2H}$$

- Query-to-context attention (which context words are relevant to some question words):

$$\beta_i = \text{softmax}_i(\max_{j=1}^M(S_{i,j})) \in \mathbb{R}^N \quad \mathbf{b} = \sum_{i=1}^N \beta_i \mathbf{c}_i \in \mathbb{R}^{2H}$$

BiDAF: Modeling and output layers



The final training loss is

$$\mathcal{L} = -\log p_{\text{start}}(s^*) - \log p_{\text{end}}(e^*)$$

Modeling layer: pass to another two layers of bi-directional LSTMs

- Attention layer is modeling interactions between query and context
- Modeling layer is modeling interactions within context words

$$\mathbf{m}_i = \text{BiLSTM}(\mathbf{g}_i) \in \mathbb{R}^{2H}$$

Output layer: two classifiers predicting the start and end positions:

$$p_{\text{start}} = \text{softmax}(\mathbf{w}_{\text{start}}^T [\mathbf{g}_i; \mathbf{m}_i]) \quad p_{\text{end}} = \text{softmax}(\mathbf{w}_{\text{end}}^T [\mathbf{g}_i; \mathbf{m}'_i])$$

$$\mathbf{m}'_i = \text{BiLSTM}(\mathbf{m}_i) \in \mathbb{R}^{2H} \quad \mathbf{w}_{\text{start}}, \mathbf{w}_{\text{end}} \in \mathbb{R}^{10H}$$

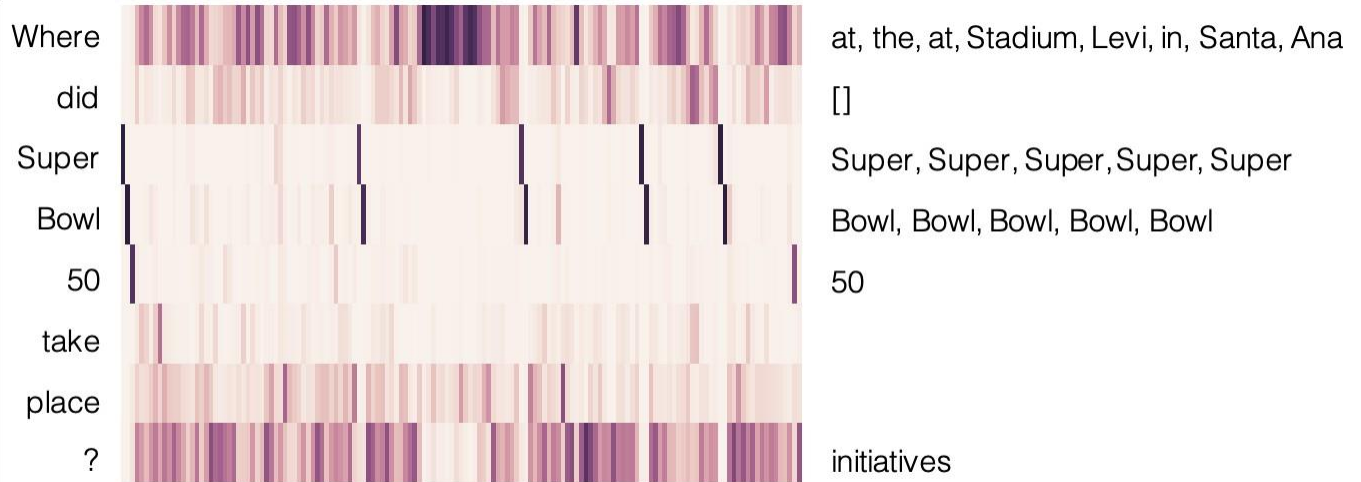
BiDAF: Performance on SQuAD

- F1=77.3 % on SQuAD v1.1.
- Without context-to-query attention 67.7 F1
- Without query-to-context attention 73.7 F1
- Without character embedding 75.4 F1

	Published ¹²	LeaderBoard ¹²
Single Model	EM / F1	EM / F1
LR Baseline (Rajpurkar et al., 2016)	40.4 / 51.0	40.4 / 51.0
Dynamic Chunk Reader (Yu et al., 2016)	62.5 / 71.0	62.5 / 71.0
Match-LSTM with Ans-Ptr (Wang & Jiang, 2016)	64.7 / 73.7	64.7 / 73.7
Multi-Perspective Matching (Wang et al., 2016)	65.5 / 75.1	70.4 / 78.8
Dynamic Coattention Networks (Xiong et al., 2016)	66.2 / 75.9	66.2 / 75.9
FastQA (Weissenborn et al., 2017)	68.4 / 77.1	68.4 / 77.1
BiDAF (Seo et al., 2016)	68.0 / 77.3	68.0 / 77.3
SEDT (Liu et al., 2017a)	68.1 / 77.5	68.5 / 78.0
RaSoR (Lee et al., 2016)	70.8 / 78.7	69.6 / 77.7
FastQAExt (Weissenborn et al., 2017)	70.8 / 78.9	70.8 / 78.9
ReasonNet (Shen et al., 2017b)	69.1 / 78.9	70.6 / 79.4
Document Reader (Chen et al., 2017)	70.0 / 79.0	70.7 / 79.4
Ruminating Reader (Gong & Bowman, 2017)	70.6 / 79.5	70.6 / 79.5
jNet (Zhang et al., 2017)	70.6 / 79.8	70.6 / 79.8
Conductor-net	N/A	72.6 / 81.4
Interactive AoA Reader (Cui et al., 2017)	N/A	73.6 / 81.9
Reg-RaSoR	N/A	75.8 / 83.3
DCN+	N/A	74.9 / 82.8
AIR-FusionNet	N/A	76.0 / 83.9
R-Net (Wang et al., 2017)	72.3 / 80.7	76.5 / 84.3
BiDAF + Self Attention + ELMo	N/A	77.9 / 85.3
Reinforced Mnemonic Reader (Hu et al., 2017)	73.2 / 81.8	73.2 / 81.8

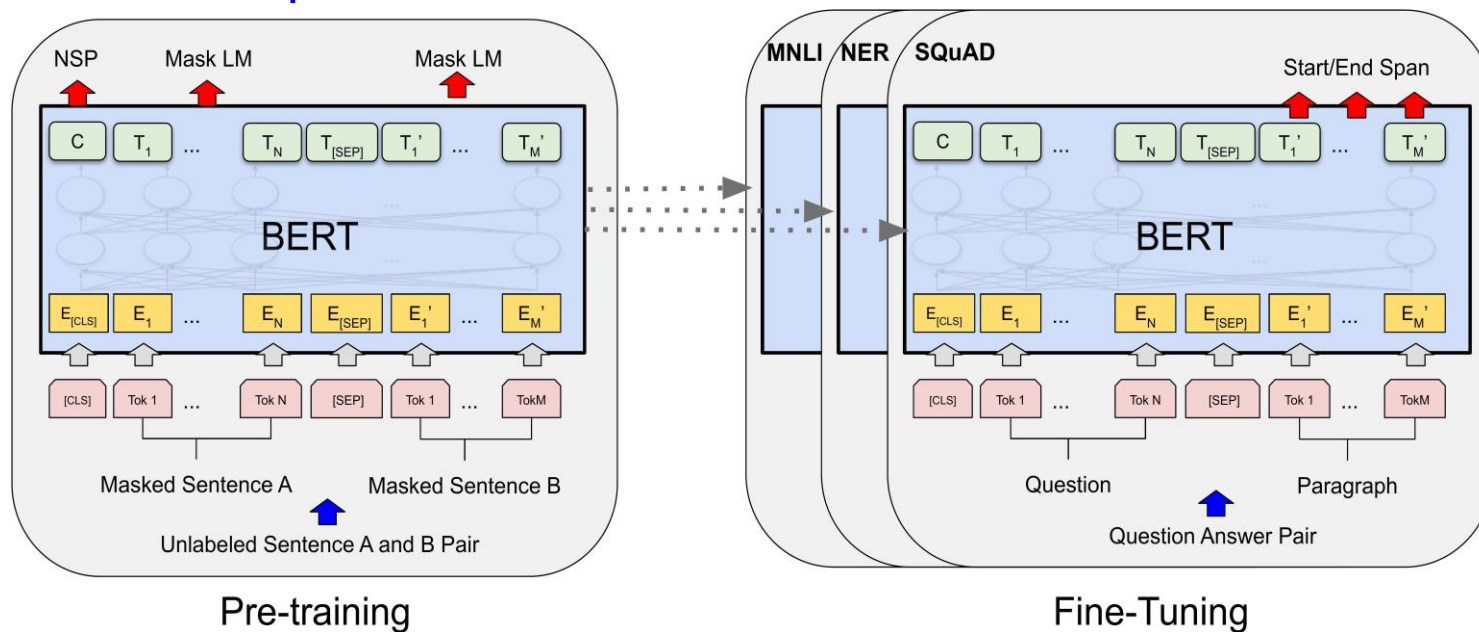
Attention visualization

Super Bowl 50 was an American football game to determine the champion of the National Football League (NFL) for the 2015 season . The American Football Conference (AFC) champion Denver Broncos defeated the National Football Conference (NFC) champion Carolina Panthers 24–10 to earn their third Super Bowl title . The game was played on February 7, 2016 , **at Levi 's Stadium in the San Francisco Bay Area at Santa Clara , California** . As this was the 50th Super Bowl , the league emphasized the " golden anniversary " with various gold-themed initiatives , as well as temporarily suspending the tradition of naming each Super Bowl game with Roman numerals (under which the game would have been known as " Super Bowl L ") , so that the logo could prominently feature the Arabic numerals 50 .

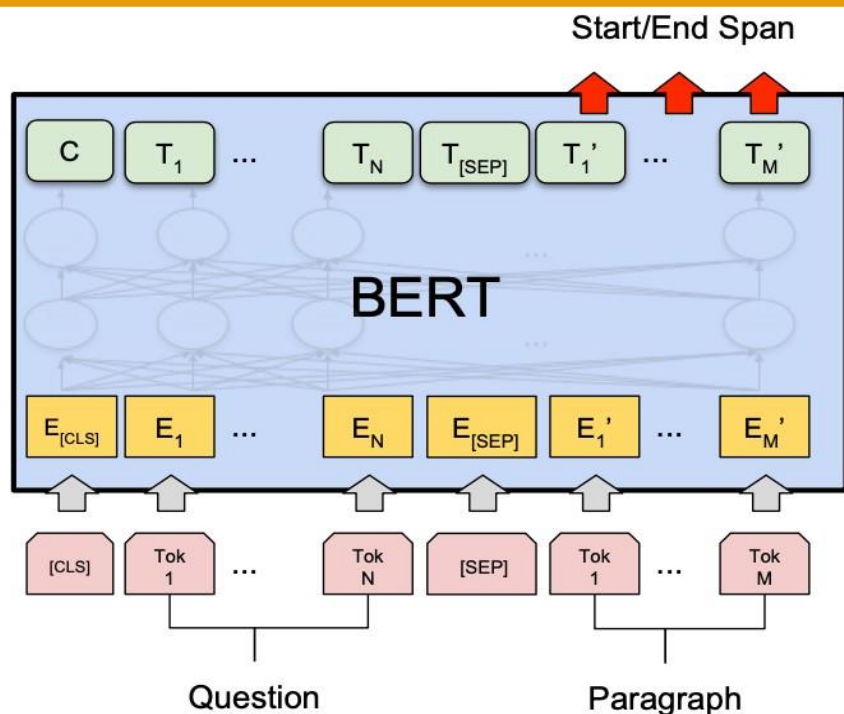


BERT for reading comprehension

- BERT is a deep bidirectional Transformer encoder pre-trained on large amounts of text (Wikipedia + BooksCorpus)
- BERT is pre-trained on two training objectives:
 - Masked language model (MLM)
 - Next sentence prediction (NSP)
- BERTbase has 12 layers and 110M parameters, BERTlarge has 24 layers and 330M parameters



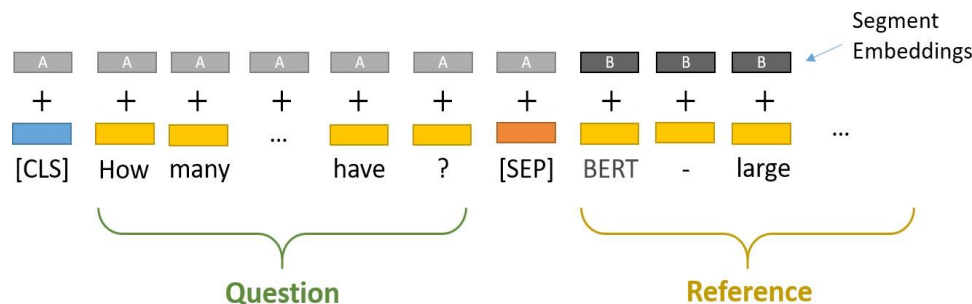
BERT for reading comprehension



Question = Segment A

Passage = Segment B

Answer = predicting two endpoints in segment B



Question: How many parameters does BERT-large have?

Reference Text: BERT-large is really big... it has 24 layers and an embedding size of 1,024, for a total of 340M parameters! Altogether it is 1.34GB, so expect it to take a couple minutes to download to your Colab instance.

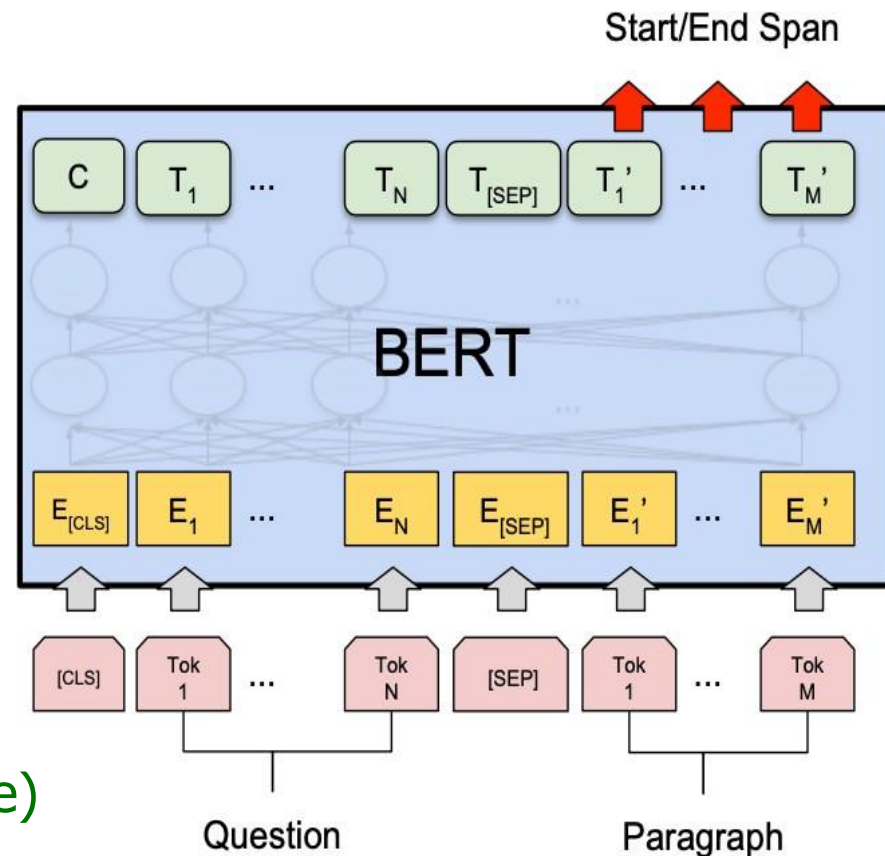
Image credit: <https://mccormickml.com/>

BERT for reading comprehension

$$\mathcal{L} = -\log p_{\text{start}}(s^*) - \log p_{\text{end}}(e^*)$$

	F1	EM
Human performance	91.2	82.3
BiDAF	77.3	67.7
BERT-base	88.5	80.8
BERT-large	90.9	84.1
XLNet	94.5	89.0
RoBERTa	94.6	88.9
ALBERT	94.8	89.3

(dev set, except for human performance)



Question Answering Datasets

  paperswithcode.com/task/question-answering

Datasets



MS COCO



MNIST



Megan Brianna Crain 9855156969 God of Creation



Natural Questions



MS MARCO



HellaSwag



TriviaQA



ConceptNet



HotpotQA



PIQA

[See all 405 question answering datasets](#)

Subtasks



Open-Domain Question Answering



Open-Ended Question Answering



Knowledge Base Question Answering



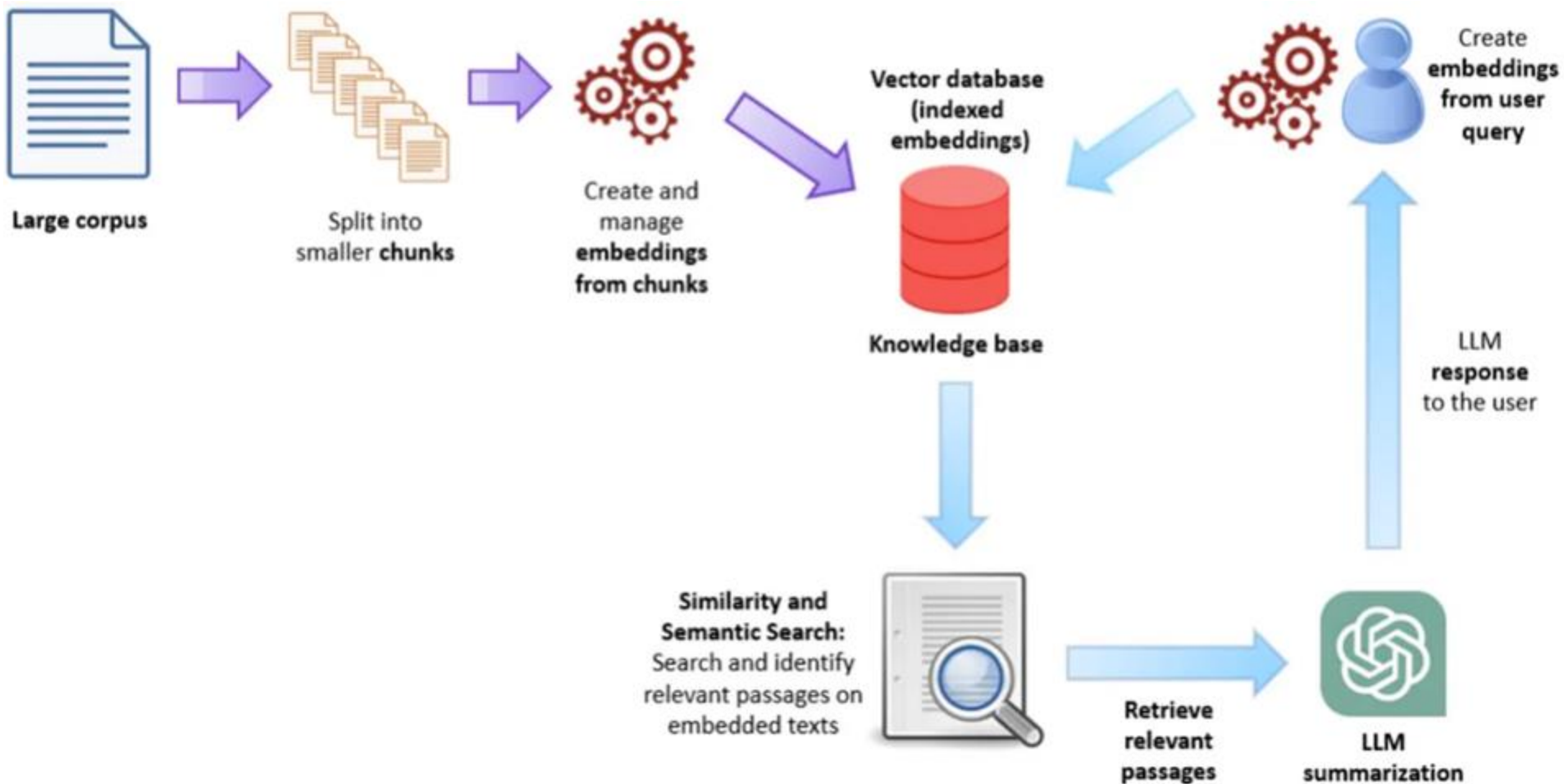
Zero-Shot Video Question Answer

[Show all 19 subtasks](#)

- 1. Passage Ranking:** Ranking passages based on the query.
 - **Scale:** Approximately 8.8 million passages from popular websites.
- 2. Document Ranking:** Ranking documents based on the query.
 - **Scale:** 3.2 million documents.
- 3. Question Answering:** Answering questions using the provided passages or documents.
 - **Scale:** 100,000 real user questions from the Bing search engine, accompanied by short extracted answers.
- 4. Conversational Search:** Ongoing conversational search, where each query carries contextual information from previous questions.

- Launched in 11/2022 by OpenAI
- ChatGPT stands for Generative Pre-Trained Transformer. It can resemble human-like responses
- Data: from online databases, containing books, webtexts, Wikipedia, articles, and other online literature.
- Usages: chatbots, virtual assistants, other applications requiring a high level of natural language processing ability.
- Anyone can use ChatGPT by integrating it into their own apps

Architecture for QA system based on RAG



• Information Retrieval:

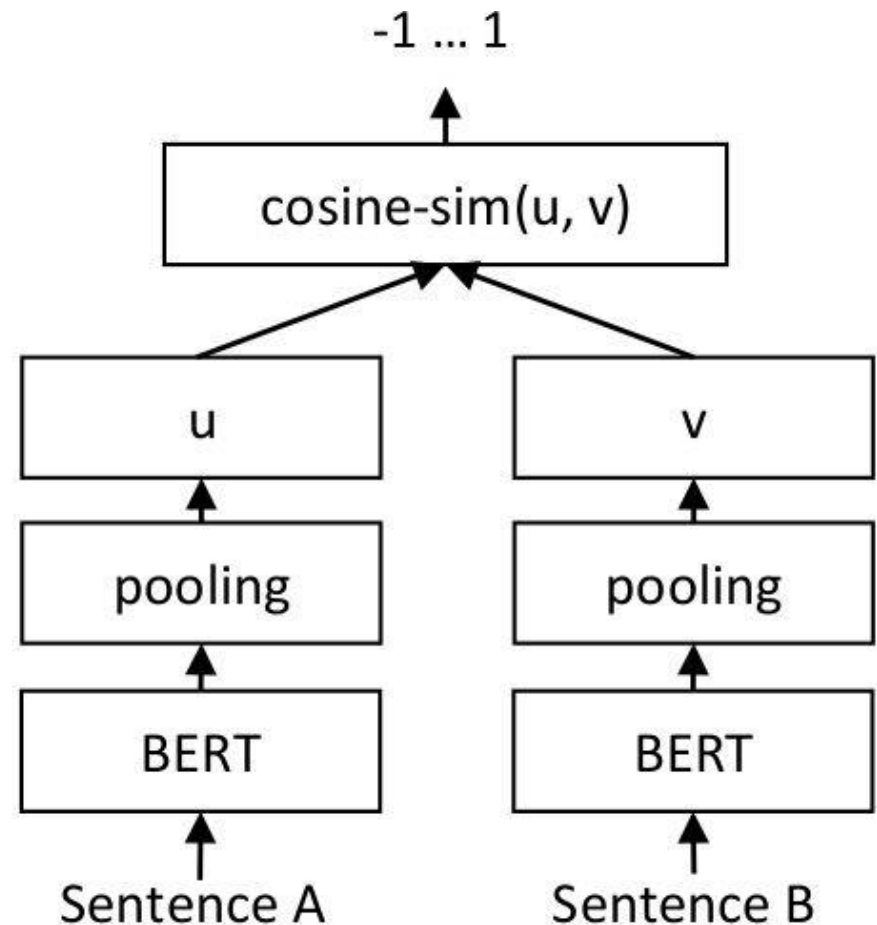
- Traditional word-matching: BM25 and variants
 - Advantages: fast and simple, no training procedures
 - Disadvantages: acronyms, paraphrases...
- Semantic searching: bi-encoder (dual-encoder), cross-encoder...
 - Advantages: “Truly understands” semantic meaning
 - Disadvantages: costly (data, time, computational and maintaining resources)

• Answer generator:

- Traditional methods rely on predefined rules and patterns for retrieving relevant documents and extracting answers.
 - Limitation: struggle to adapt to new data patterns and changes.
 - Modern methods: base on LLM (e.g., Gemini, Llama3), generate answers based on text comprehension rather than extraction.
 - Challenges: high computational costs, large training data, hallucinations.
- Using RAG

Sentence-BERT

- A modification of the pretrained BERT, use Siamese and triplet network structures to derive semantically sentence embeddings that can be compared using cosine-similarity
- SBERT can be used for tasks which are computationally not feasible to be modeled with BERT. For example, clustering of 10,000 sentences with hierarchical clustering requires with BERT about 65 hours, as around 50 million sentence combinations must be computed. With SBERT, we were able to reduce the effort to about 5 seconds



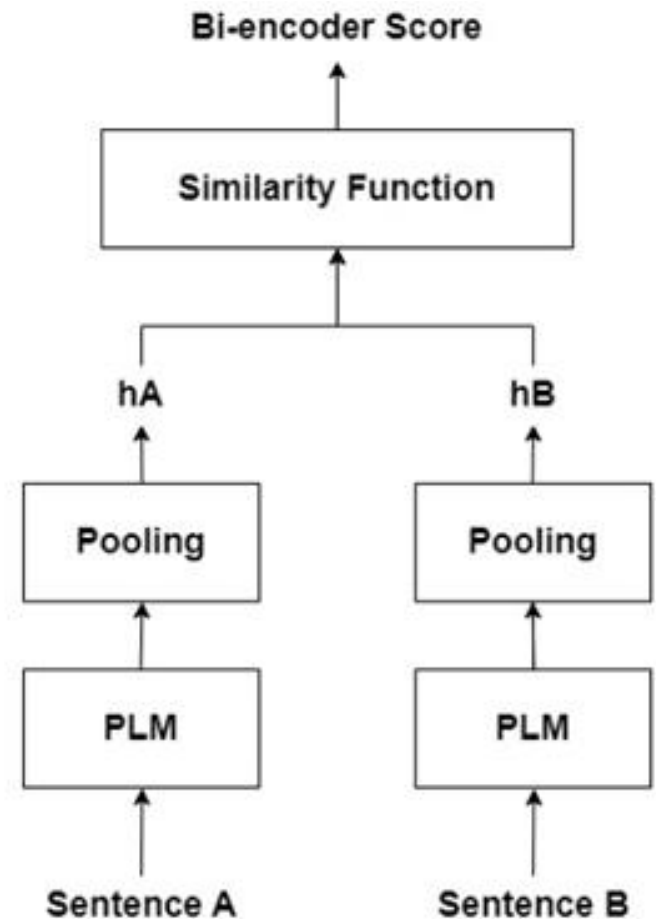
Reimers and Gurevych. [Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks](https://arxiv.org/abs/1908.10084), 2019

<https://www.sbert.net/>

- **Sentence Transformer:** a Python library that makes it easy to use SBERT models for solving tasks such as:
 - Semantic Search
 - Text Similarity
 - Clustering

Bi encoder

- is an encoder neural network architecture used in Transformer-based models (such as BERT, RoBERTa, Sentence Transformer).
- It is designed to represent text segments as **independent embedding vectors**, optimized for tasks like similarity matching, semantic search, and information retrieval.

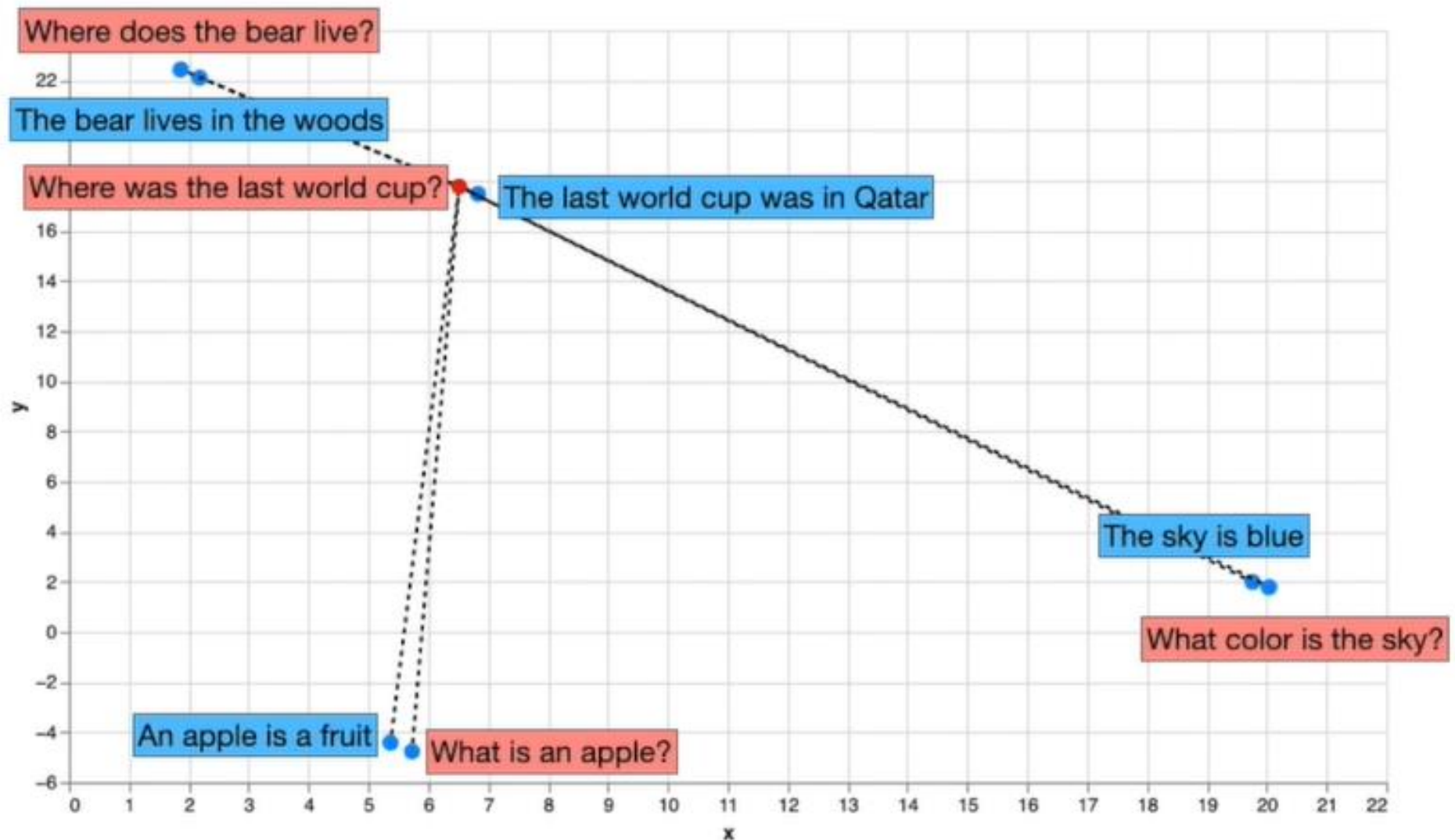


Example

```
pip install sentence-transformers
```

```
from sentence_transformers import SentenceTransformer, util
# Loading the bi-encoder model from Sentence Transformer
model = SentenceTransformer('all-MiniLM-L6-v2')
# List of documents
documents = [
    "The capital of France is Paris.",
    "Machine learning enables computers to learn from data.",
    "Artificial intelligence is the future.",
    "Paris is known for its art and culture."
]
# Encoding documents into vector embedding
doc_embeddings = model.encode(documents, convert_to_tensor=True)
# Query
query = "What is the capital of France?"
query_embedding = model.encode(query, convert_to_tensor=True)
# Calculate cosine similarity between queries and documents
scores = util.cos_sim(query_embedding, doc_embeddings)
# Find documents with the highest similarity
best_match_idx = scores.argmax().item()
print(f"The most suitable documents: {documents[best_match_idx]}")
```


Nearest neighbors: good but slow



Solution: Inverted file index

Steps in Building a Semantic Inverted Index:

1. Generate Embeddings:

- Each document/query is transformed into a **vector** representation using a **pre-trained language model** such as **BERT, Sentence Transformers, or PhoBERT**.
- Example: The sentence *"apple is a fruit"* might be mapped to a vector [0.1, 0.3, 0.7, ...].

2. Assign Semantic Tokens: Since embeddings are continuous, they are **quantized or clustered** to make them searchable.

- Example: An embedding could be assigned to **cluster ID 42** or **semantic token** representing similar vectors.

Building a Semantic Inverted Index:

3. Build the Inverted Index:

- Instead of indexing raw words, the index now maps semantic tokens or cluster IDs to the documents they appear in.
- Example:
 - Cluster 42 $\rightarrow$ [Doc A, Doc C, Doc F]
 - Cluster 51 $\rightarrow$ [Doc B, Doc D]

4. Store Metadata:

- (optionally) Store metadata (e.g., vector norms or top-k embeddings) to enable faster similarity scoring during retrieval.

How Semantic Retrieval Works

- Query Embedding Generation:

- When a user submits a query (e.g., "find fruits related to apple"), the query is transformed into an embedding vector.

- Quantization / Nearest Neighbor Search:

- The query vector is mapped to the closest cluster IDs or semantic tokens in the inverted index.
- Example: Query embedding $\rightarrow$ Cluster 42 (where similar "fruit-related" embeddings reside).

- Search the Inverted Index:

- The system retrieves the documents corresponding to the relevant clusters (e.g., "Cluster 42 $\rightarrow$ [Doc A, Doc C, Doc F]").

- Rank the Results:

- Finally, the documents are re-ranked based on the cosine similarity between the query vector and the document embeddings.

Example

Given documents:

"The capital of France is Paris.",
"The tallest mountain in the world is Mount Everest.",
"Python is a popular programming language.",
"AI is transforming the world.",
"Blockchain is a distributed ledger technology."

Given query:

"What is the capital of France?"

Write a program that performs the following tasks:

1. Represent input documents as vectors
2. Index those documents
3. Search for documents that match a given query using a bi-encoder

Example

```
!pip install faiss-cpu sentence-transformers
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# 1. Represent documents and queries into vectors
model = SentenceTransformer('paraphrase-MiniLM-L6-v2') #
using Bi-encoder, can be replaced with PhoBERT if needed
# Document data (a list of texts)
documents = [
    "The capital of France is Paris.",
    "The tallest mountain in the world is Mount Everest.",
    "Python is a popular programming language.",
    "AI is transforming the world.",
    "Blockchain is a distributed ledger technology."
]

# Encoding documents into vector embedding
doc_embeddings = model.encode(documents,
convert_to_tensor=False)
```

Example

2. Index documents by FAISS

```
d = doc_embeddings.shape[1] # Embedding dimension
index = faiss.IndexFlatL2(d) # Using IndexFlatL2 with
Euclidean distance
index.add(np.array(doc_embeddings, dtype='float32')) # Adding
vectors to the index
print(f"Indexed {index.ntotal} document.")
```

3. Search for documents based on queries

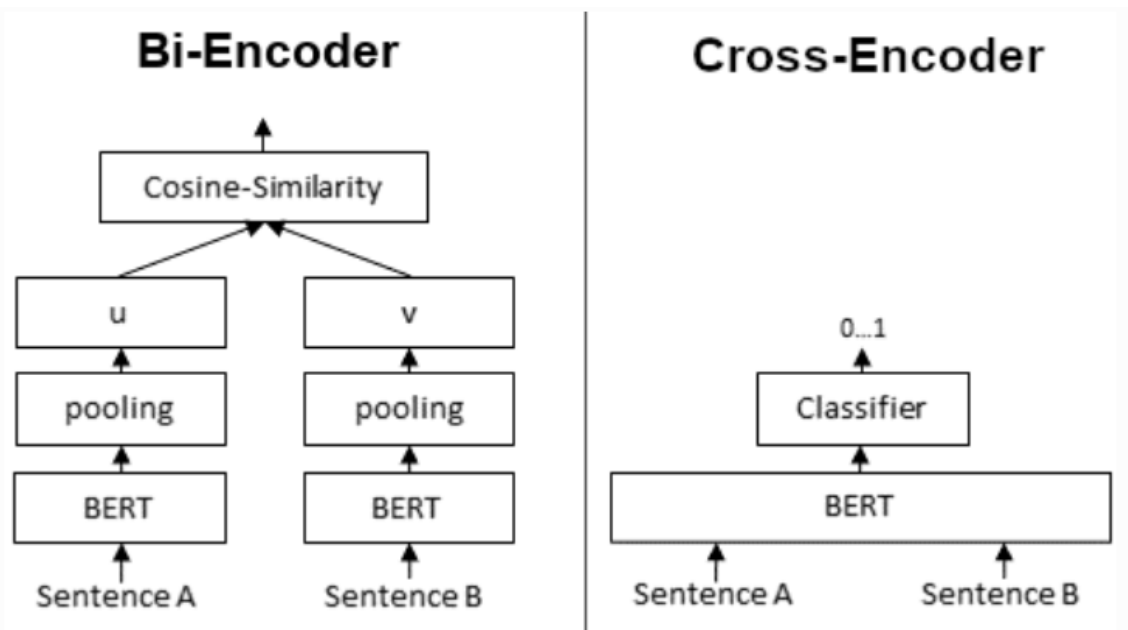
```
query = "What is the capital of France?" # Query
query_embedding = model.encode(query,
convert_to_tensor=False).reshape(1, -1) # Query encoding
k = 3 # Number of Nearest Documents to Find
distances, indices = index.search(np.array(query_embedding,
dtype='float32'), k)
```

4. Print search results

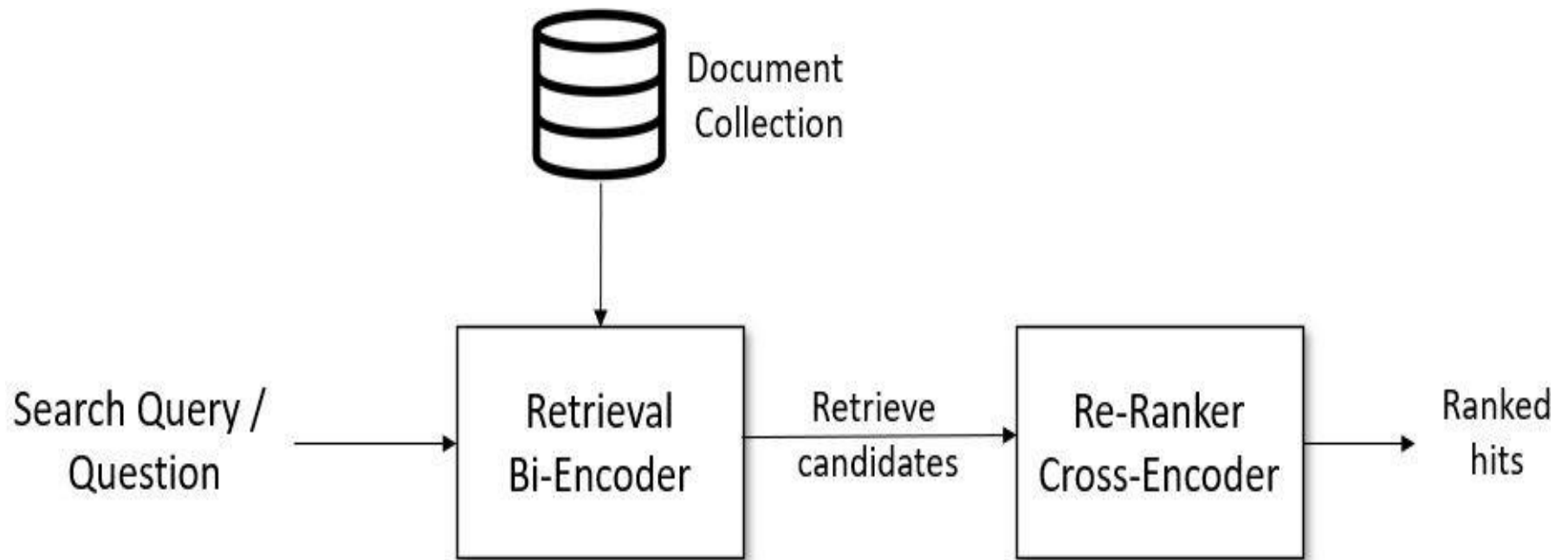
```
print("\n Search results:")
for i, idx in enumerate(indices[0]):
    print(f"Document {i+1}: {documents[idx]} (Distance:
{distances[0][i]:.4f})")
```

Limitations of Bi-Encoder

- Bi-encoder optimizes speed but accepts a reduction in accuracy
 - Only computes cosine similarity between pre-encoded vectors.
 - Does not fully capture the complex semantic relationships between the query and the document.
- Cross-encoders look at contexts, helping to determine relevance in more detail.
- Bi-encoders are often used to quickly retrieve a top-K list of relevant documents from a large data set. Reranking then improves the accuracy of this top-K, ensuring that the most relevant documents are ranked higher.



Retrieve and Re-rank



<https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>

Cross-encoder

- The query and document are concatenated into a single input sequence, typically separated by a special token (e.g., [SEP] in BERT).
- The cross-encoder computes a relevance score between the query and the document.
- **Advantage:**
 - Since the query and document are not encoded separately, the cross-encoder can accurately consider the specific context between words in both, allowing it to better distinguish documents with the highest semantic similarity.
- **Disadvantage:**
 - Computation is slow.

Contrastive learning

- **Contrastive Learning** is a technique that aims to learn vector embeddings such that similar data points are close together, while unrelated data points are pushed farther apart.
- **Data samples** are organized into pairs:
 - **Positive Pair:** Two data samples that are identical or semantically related (e.g., the same sentence with random augmentations, or two sentences expressing the same meaning).
 - **Negative Pair:** Two data samples that are unrelated or semantically different.

- **Loss Function:**

- **Contrastive Loss:** Minimizes the distance between positive pairs and maximizes the distance between negative pairs.
- **Triplet Loss:** Based on three samples (anchor, positive, and negative), it optimizes so that the positive is closer to the anchor than the negative.
- **InfoNCE Loss** (Information Noise-Contrastive Estimation Loss): A type of cross-entropy loss that optimizes the separation of positive samples from negatives within a large batch.

- **Data Augmentation:**

- Generate additional data from the original while preserving its meaning.
- For example: removing words, changing sentence structure, etc.

- SimCSE (Simple Contrastive Learning of Sentence Embeddings)
- **Idea:** Train the model to pull similar sentences closer together and push dissimilar ones farther apart.
 - **Unsupervised SimCSE:** Uses dropout (randomly dropping some neurons in the network) to create slightly different representations of the same sentence.
 - **Supervised SimCSE:** Similar sentence pairs are taken from bilingual data or paraphrase datasets.
- **Goal:** Improve the quality of sentence-level semantic representations, based on pretrained language models such as BERT, RoBERTa, and GPT.

Gao, Tianyu, Xingcheng Yao, and Danqi Chen. "Simcse: Simple contrastive learning of sentence embeddings." *EMNLP* (2021).

- InfoNCE loss:

Similarity function Sentence embedding Temperature

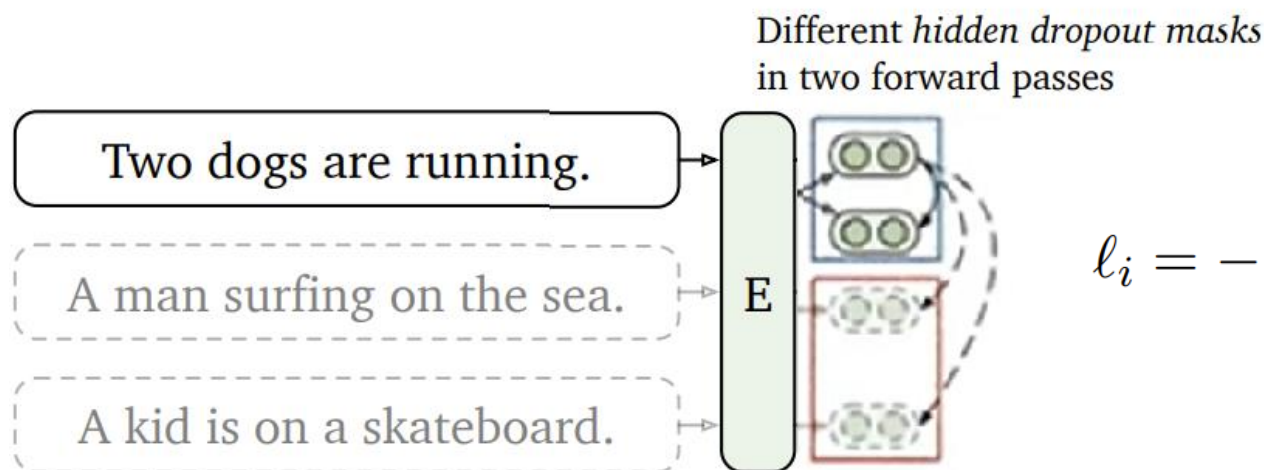
$\ell_i = -\log \frac{e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_i^+)/\tau}}{\sum_{j=1}^N e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_j^+)/\tau}}$

← Positive pairs

← Negative pairs

Unsupervised SimCSE

- **Positive sample:** embedding of the same sentence, using different dropout masks $\rightarrow$ produce similar sentence pairs
- **Negative sample:** embedding of other sentences in the same batch



$$\ell_i = -\log \frac{e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_i^+)/\tau}}{\sum_{j=1}^N e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_j^+)/\tau}}$$

Supervised SimCSE

- Use labeled data to improve performance quality. Positive pairs are derived from bilingual data or from paraphrase datasets.
- The model learns to distinguish between similar and dissimilar sentences based on available labels.
- **Loss function:** Cross-entropy loss is commonly used to optimize the difference between labeled sentence pairs.

Given one premise,

- Premise: *There are two dogs running.*

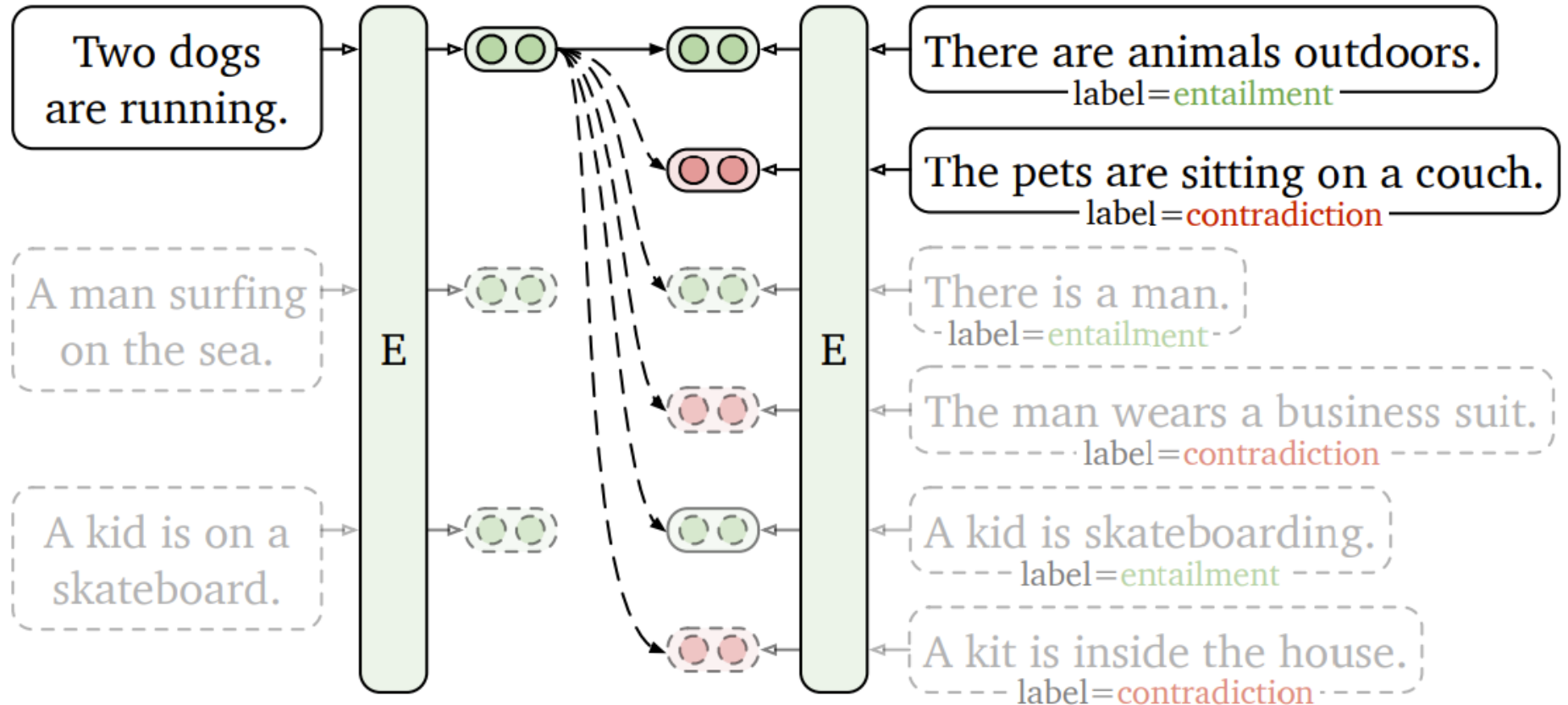
Annotators are required to write hypotheses of

- Entailment: *There are animals outdoors.*
- Contradiction: *The pets are sitting on a couch.*
- Neutral: *The dogs are catching a ball.*

Positive pairs

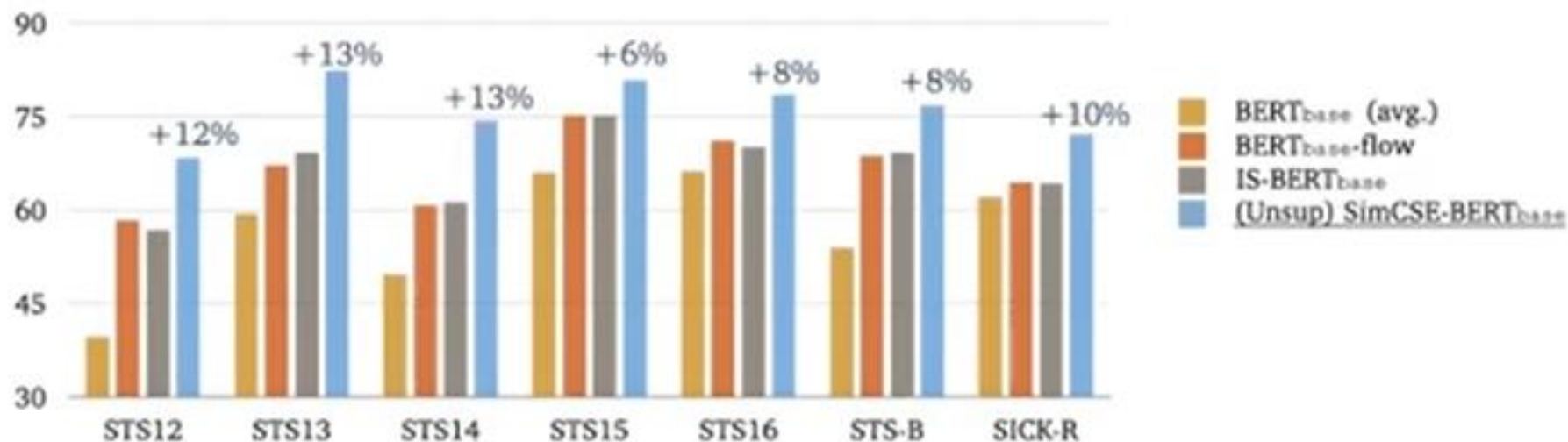
Hard negatives

Supervised SimCSE



Unsupervised SimCSE

Semantic textual similarity (STS) tasks: Spearman's correlation

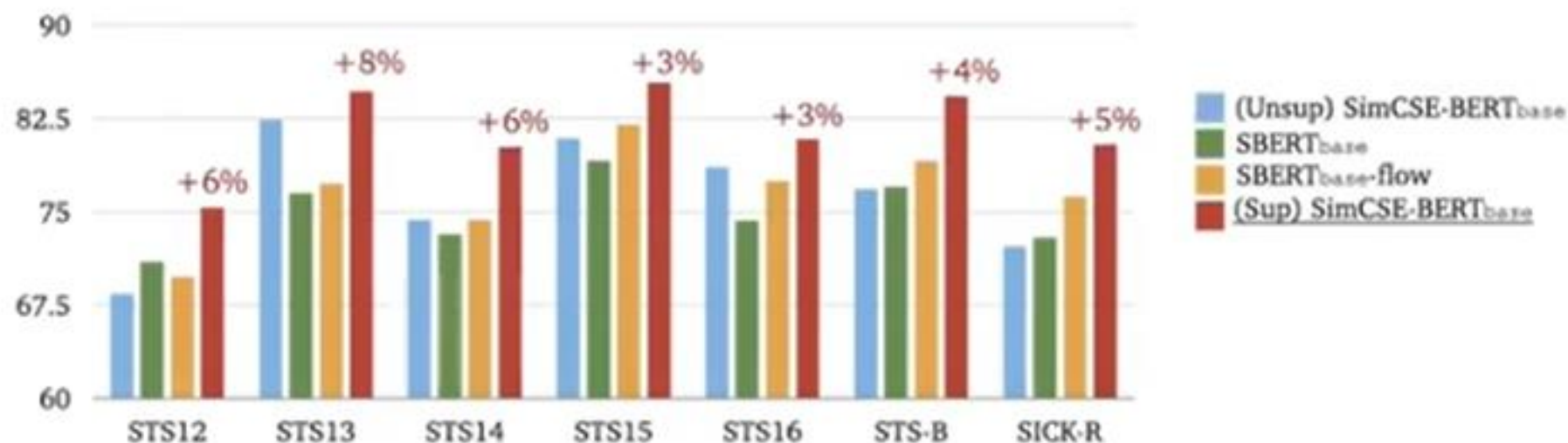


~10% higher than previous SOTA

~20% higher than avg. BERT embeddings

Supervised SimCSE

Semantic textual similarity (STS) tasks: Spearman's correlation



Even unsupervised SimCSE matches supervised SentenceBERT
6.7% higher than SentenceBERT using the same NLI datasets

Different unsupervised positive pairs

Different dropout

- SimCSE (dropout)

NLP is interesting. — NLP is interesting.

Compare it to

- Next sentence
- Synonym replacement
- Crop
- Delete one word

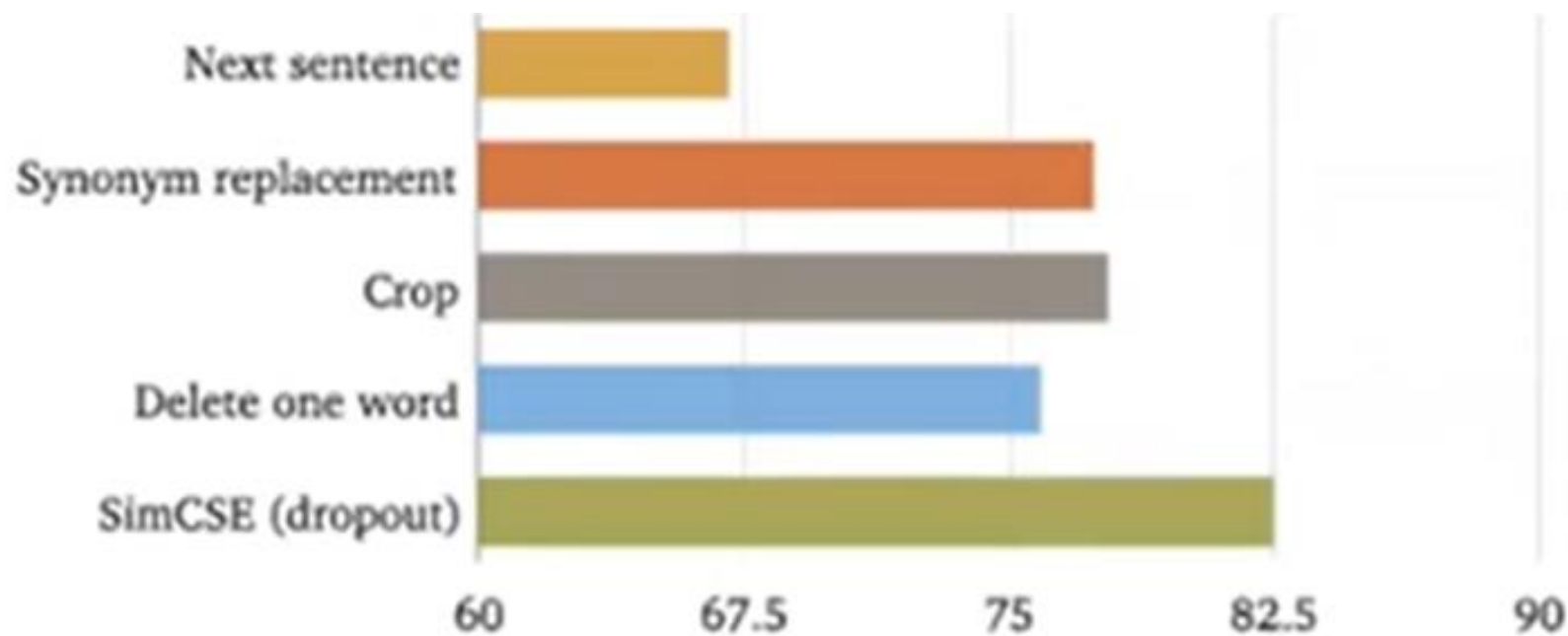
I do NLP. — NLP is interesting.

The movie is great. — The movie is fantastic.

~~Two dogs~~ are running. — ~~Two~~ dogs are running.

Two dogs are running. — Two dogs are running.

Unsupervised SimCSE

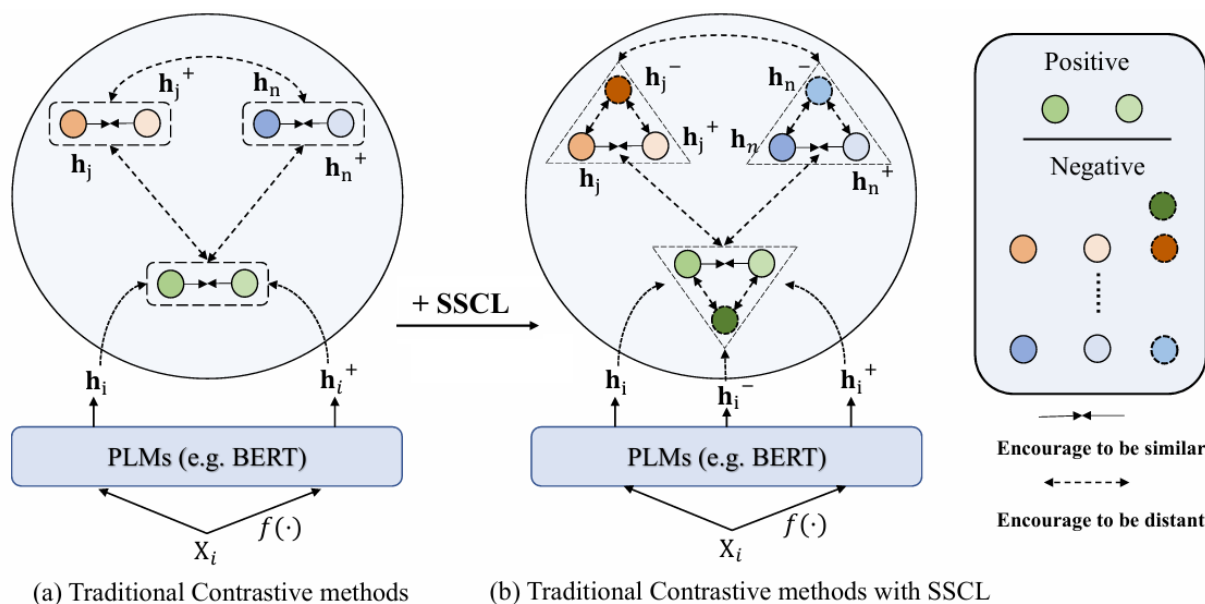


Default setting: 1 million sentences randomly sampled from English Wikipedia, N=64, evaluated on STS-B development set (Spearman's correlation)

- Predicting sentence itself $\gg$ predicting next sentence
- Using dropout as data augmentation $\gg$ discrete data augmentation (!!)

Self-Contrastive Learning (SSCL)

- In unsupervised sentence representation, pre-trained language models (PLMs) and contrast learning showed positive results.
- Over-smoothing problem: sentence representations between two adjacent layers are significantly similar as the number of coding layers increases deeper
- New contrast learning strategy: use sentence representations in the M intermediate class of PLMs as negative samples



Chen et al.. [Alleviating Over-smoothing for Unsupervised Sentence Representation](#). ACL 2023

Self-Contrastive Learning (SSCL)

- Mẫu dương: (h_i, h_i^+) được tạo ra bằng 1 phương pháp tăng cường dữ liệu nào đó
- Mẫu âm (h_i, h_i^-)
- $\Psi(\cdot)$: cosin similarity

$$\mathcal{L}_{hne} = -\log \frac{\exp(\Psi(\mathbf{h}_i, \mathbf{h}_i^+)/\tau)}{\sum^N ((\exp(\Psi(\mathbf{h}_i, \mathbf{h}_j)/\tau) + \exp(\Psi(\mathbf{h}_i, \mathbf{h}_i^-)/\tau))$$

Initial Negative Samples in Batch

Intermediate negative samples

<i>Base Version</i>								
GloVe embeddings (avg.) [♡]	55.14	70.66	59.73	68.25	63.66	58.02	53.76	61.32
BERT (cls.)	29.70	49.38	39.67	56.03	56.19	43.87	52.06	46.70
SSCL-BERT (cls.)	49.21	67.59	58.96	69.94	68.00	62.87	60.43	62.42
BERT (avg.)	48.26	47.72	46.83	52.30	59.88	54.27	56.41	52.24
SSCL-BERT (avg.)	53.93	63.10	56.41	68.00	70.46	64.85	61.15	62.56
BERT-flow [♡]	58.40	67.10	60.85	75.16	71.22	68.66	64.47	66.55
BERT-whitening [♡]	57.83	66.90	60.90	75.08	71.31	68.24	63.73	66.28
IS-BERT [♡]	56.77	69.24	61.21	75.23	70.16	69.21	64.25	66.58
CT-BERT [♡]	61.63	76.80	68.47	77.50	76.48	74.31	69.19	72.05
SimCSE	68.40	82.41	74.38	80.91	78.56	76.85	72.23	76.25
SSCL-SimCSE	71.68	83.50	76.42	83.46	78.39	79.03	71.76	77.90
<i>Large Version</i>								
BERT _{large} (cls.)	33.06	57.64	47.95	55.83	62.42	49.66	53.87	51.49
BERT _{large} -flow	65.20	73.39	69.42	74.92	77.63	72.26	62.50	70.76
BERT _{large} -whitening	64.35	74.64	69.64	74.68	75.94	60.81	72.47	70.35
Consert _{large}	70.69	82.96	74.13	82.78	76.66	77.53	70.47	76.45
SimCSE _{large}	69.17	84.36	75.09	83.99	78.61	79.54	71.97	77.53
SSCL-SimCSE_{large}	71.98	85.74	77.94	85.94	80.08	81.20	74.28	79.69